Informe del punto implementado

Escritura y lectura de archivos grandes, batching, buffer, procesamiento paralelo, legibilidad, mantenibilidad, PEP 8 y API web

> **Proyecto:** Buscador Eficiente de Información
> **Repositorio:** pef-26
> **Fecha:** 12 de octubre de 2026

1. Objetivo del trabajo

El objetivo fue extender el buscador para que demostrara los conceptos trabajados en clase: manejo de archivos, uso eficiente de recursos, organización del código y exposición del motor mediante un framework web.

El sistema recibe una consulta, busca los términos dentro de un corpus de archivos de texto y devuelve los documentos más relevantes. La solución conserva el buscador existente y agrega mejoras relacionadas con la lectura, el procesamiento y la conexión mediante HTTP.

2. Estructura general del proyecto

Archivo	Responsabilidad
`src/baseline.py`	Versión secuencial de referencia. Lee cada archivo y busca por fuerza bruta.
`src/optimizado.py`	Normalización, índice invertido, búsqueda, ranking TF-IDF y caché.
`src/concurrente.py`	Construcción del índice por lotes usando varios procesos.
`src/api.py`	API FastAPI con el endpoint `GET /buscar`.
`tests/test_api.py`	Pruebas del endpoint HTTP.
`requirements-dev.txt`	Dependencias de desarrollo y de la API.

3. Lectura de archivos grandes y buffer

¿Qué problema había?

El buscador ya leía el contenido completo de cada documento, pero no declaraba un buffer explícito. Al trabajar con muchos archivos o archivos grandes, es importante controlar cómo se realizan las operaciones de entrada y salida.

¿Qué se implementó?

En `src/optimizado.py` se agregó:

```
BUFFER_SIZE = 1024 * 1024
```

El índice secuencial ahora abre cada archivo indicando un buffer de 1 MiB:

```
with archivo.open(
   "r",
   encoding="utf-8",
   errors="ignore",
   buffering=BUFFER_SIZE,
) as entrada:
   texto = entrada.read()
```

El buffer es una zona intermedia de memoria que agrupa operaciones de lectura. Esto reduce la cantidad de accesos pequeños al sistema operativo y resulta apropiado para recorrer documentos de texto.

> **Aclaración:** el buffer no significa que todo el corpus se cargue en memoria. Se usa un documento por vez. El índice sí queda en memoria porque es precisamente la estructura que permite responder consultas rápidamente.

La misma constante se reutiliza en `src/concurrente.py`. Así ambas implementaciones usan la misma política de lectura y no se duplica configuración.

4. Batching: procesamiento por lotes

Batching significa agrupar elementos de entrada en lotes antes de procesarlos. En lugar de crear una tarea independiente para cada archivo, el programa construye lotes de documentos.

En `concurrente.py`, `_lotes_equitativos()` reparte los documentos de manera equilibrada. El resto de una división se distribuye entre los primeros lotes, por lo que ningún lote difiere en más de un elemento.

```
cantidad_lotes = procesos * LOTES_POR_PROCESO
lotes = _lotes_equitativos(tareas, cantidad_lotes)
```

Se utiliza más de un lote por proceso para que el pool pueda equilibrar mejor la carga si algunos documentos son más pesados que otros. Se evita, en cambio, crear una tarea por archivo, porque el costo de comunicación y serialización sería innecesario.

5. Procesamiento paralelo

La indexación requiere leer documentos, tokenizarlos, normalizarlos y contar frecuencias. Es una etapa costosa y se implementó con `ProcessPoolExecutor`.

1. **Particionado:** el proceso principal asigna un `doc_id` estable a cada ruta y crea los lotes.
2. **Map:** cada worker ejecuta `_procesar_lote()`, lee sus archivos y genera un índice parcial.
3. **Reduce:** el proceso principal fusiona los índices parciales mediante `_fusionar()`.

Los workers no modifican una estructura compartida. Cada uno devuelve su propio diccionario y el proceso principal integra los resultados. Esto evita locks y condiciones de carrera.

Situación	Comportamiento
Pocos archivos	Se procesa en el proceso principal porque crear workers costaría más que el trabajo.
Muchos archivos	Se divide el corpus en lotes y se procesa con varios procesos.
Un worker	Se conserva un camino secuencial útil para pruebas y comparación.

6. Índice invertido y ranking

El índice invertido guarda, para cada término, los documentos donde aparece y su frecuencia. Conceptualmente:

```
"algoritmo" -> {12: 3, 45: 1, 784: 5}
```

La consulta no necesita recorrer todos los documentos. Primero obtiene los candidatos desde el índice, luego calcula TF-IDF y finalmente devuelve los mejores `top_k` resultados usando un heap.

- La indexación se paga una vez.
- Las consultas posteriores reutilizan la estructura.
- La caché LRU evita recalcular consultas repetidas.
- El ranking es determinista: ante empate se prioriza el documento con menor `doc_id`.

7. API web con FastAPI

Se creó `src/api.py` para convertir el buscador en un servicio conectable desde un navegador, otra aplicación o un frontend.

Endpoint

```
GET /buscar?q=algoritmo&top_k=10
```

- `q`: consulta obligatoria, con al menos un carácter.
- `top_k`: cantidad máxima de resultados, entre 1 y 100.

La respuesta tiene tres partes: la consulta normalizada para mostrar, el total de coincidencias y la lista de resultados con nombre y score.


```
{
 "consulta": "algoritmo",
 "total": 957,
 "resultados": [
   {
     "documento": "doc_000784.txt",
     "score": 0.6592783129377415
   }
 ]
}
```


Construcción diferida y reutilización

La aplicación no indexa el corpus al importar el módulo. Construye el índice en la primera consulta, lo guarda en memoria y reutiliza el objeto `Buscador` en las peticiones siguientes.

Esto evita leer y tokenizar los 1000 o 50.000 documentos cada vez que un cliente consulta la API.

Actualización automática del índice

Durante las pruebas apareció un caso real: si la API se consultaba cuando el corpus estaba vacío y luego se generaban documentos, el índice anterior seguía vacío. Para resolverlo, `api.py` calcula una firma del corpus formada por nombre, fecha de modificación y tamaño de cada archivo. Si esa firma cambia, reconstruye el índice.

Por eso ahora se pueden agregar o regenerar documentos sin tener que borrar manualmente el estado interno del buscador.

8. Legibilidad y mantenibilidad

- Se utilizaron nombres descriptivos como `BUFFER_SIZE`, `firma_corpus`, `obtener_buscador` y `procesar_lote`.
- La API está separada de la lógica de indexación: cada módulo tiene una responsabilidad clara.
- Se reutilizaron funciones y clases existentes en lugar de duplicar el algoritmo de búsqueda.
- Las funciones públicas y las partes no evidentes tienen docstrings que explican parámetros, retornos y propósito.
- La configuración del buffer está centralizada en una constante.
- Los errores se informan mediante códigos HTTP apropiados, por ejemplo `503` si no existe el corpus y `422` si la consulta es inválida.

9. PEP 8 y formato

Se ejecutó Ruff sobre los archivos modificados. Ruff verifica errores de estilo, imports, nombres, formato y reglas de documentación configuradas en `pyproject.toml`.

```
ruff check src/api.py src/optimizado.py src/concurrente.py tests/test_api.py
ruff format --check src/api.py src/optimizado.py src/concurrente.py tests/test_api.py
```

Resultado: todos los checks pasaron correctamente.

10. Pruebas y verificación

Se agregaron pruebas para confirmar el comportamiento HTTP:

- Una consulta válida devuelve total, documentos y ranking.
- Una consulta formada solo por espacios devuelve error 422.

La suite completa terminó con:

```
44 passed, 1 warning
```

La advertencia proviene de una dependencia externa usada por el cliente de pruebas de Starlette; no corresponde al código del proyecto.

11. Ejecución paso a paso

```
cd "C:\Users\jorge\Desktop\PEF\parcial2\pef-26"
python -m pip install -r requirements-dev.txt
python src\baseline.py --generar --num-docs 1000
python -m uvicorn api:app --app-dir src --reload
```

Luego se puede abrir:

```
http://127.0.0.1:8000/docs
http://127.0.0.1:8000/buscar?q=algoritmo&top_k=10
```

La documentación de `/docs` permite ejecutar el endpoint desde el navegador usando el botón **Try it out**.

12. Resultado final

Con un corpus de 1000 documentos se verificó una consulta real de `algoritmo` que devolvió 957 coincidencias y los 10 documentos con mayor score. Esto confirma que la API utiliza el índice invertido, el ranking TF-IDF y el límite `top_k` correctamente.

> **Conclusión:** el punto integra optimización de entrada/salida, batching, procesamiento paralelo, estructuras de datos eficientes, documentación, PEP 8 y una interfaz web reutilizable, manteniendo las pruebas existentes y agregando cobertura específica para la API.